

Treinamento SQL - Essentials

LUIS_DEV



Cláusulas

Cláusulas

As principais cláusulas SQL são SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, como vimos na última aula, as duas primeiras as suas principais atribuições e objetivos. Antes de nos aprofundarmos mais nas próximas cláusulas, vamos expandir as possibilidades que podem ser aplicadas à cláusula SELECT.

SELECT

Operações matemáticas:

Aplicação de operações básicas matemáticas no seus selects, como: soma, subtração, divisão e multiplicação, então é possível aplicar operações que possam criar novos campos a partir da aplicação das operações matemáticas.



Cláusulas

Cláusulas - SELECT

Operações matemáticas:

Símbolo dos operadores:

- Soma(+)
- Subtração(-)
- Divisão(/)
- Multiplicação(*)

Exemplo:

```
select id_instrutor, salario * 12 as salarial_anual from graduacao.instrutor;
```

```
select id_instrutor, salario + 1500 as aumento_salarial from graduacao.instrutor;
```

```
select id_instrutor, salario - 7000 as subtracao_salarial from graduacao.instrutor;
```

```
select id_instrutor, salario / 3 as aumento_salarial from graduacao.instrutor;
```



Cláusulas

Cláusulas - SELECT

Distinct

O uso do DISTINCT tem como objetivo eliminar a duplicidade de dados a partir de um determinado campo selecionado. O DISTINCT pode ser combinado com instruções de agrupamentos, com grupos de colunas também.

Exemplos:

```
select distinct id_aluno, ano from graduacao.nota_aluno where id_aluno = '13403';
```

```
select *distinct id_aluno from graduacao.nota_aluno;
```



Cláusulas

Cláusulas - SELECT

Case

A instrução Case tem como objetivo gerar uma coluna a partir de condições estabelecidas, gerando valores colunares de acordo com a premissa desenhada para cada condição gerada dentro do CASE.

Exemplos:

```
select i.*,  
case  
when salario < 40000 then 'professor substituto'  
when salario <= 70000 then 'professor pleno'  
when salario < 100000 then 'professor sênior'  
else 'professor adjunto'  
end cargo_instrutor  
from graduacao.instrutor i;
```



Cláusulas

Cláusulas - SELECT

Concat

A instrução Concat tem como objetivo gerar a junção de strings formando uma nova coluna a partir de outras colunas ou textos.

Exemplos:

```
select
j.*,
cargo_instrutor + ' - ' + nome_departamento as cargo_departamento
from(
select i.*,
case
when salario < 40000 then 'professor substituto'
when salario <= 70000 then 'professor pleno'
when salario < 100000 then 'professor sênior'
else 'professor adjunto'
end cargo_instrutor
from graduacao.instrutor i) j;
```

```
select
j.*,
concat(cargo_instrutor, ' - ', nome_departamento) as cargo_departamento
from(
select i.*,
case
when salario < 40000 then 'professor substituto'
when salario <= 70000 then 'professor pleno'
when salario < 100000 then 'professor sênior'
else 'professor adjunto'
end cargo_instrutor
from graduacao.instrutor i) j;
```



Cláusulas

Cláusulas - SELECT

Distinct:

O uso do DISTINCT tem como objetivo eliminar a duplicidade de dados a partir de um determinado campo selecionado. O DISTINCT pode ser combinado com instruções de agrupamentos, com grupos de colunas também.

Exemplos:

```
select distinct id_aluno, ano from graduacao.nota_aluno where id_aluno = '13403';
```

```
select *distinct id_aluno from graduacao.nota_aluno;
```



Cláusulas

WHERE

A cláusula WHERE é utilizada para gerar filtros de dados nas operações no banco de dados, como nos SELECT, UPDATE, INSERT e DELETE. Com isso, é possível melhorar a performance das consultas, pois apenas uma parte dos dados totais será trazido em um select, ou deletado... O WHERE garante que apenas uma parte dos dados vai ser impactada em caso de erros ou deleções e atualizações indevidas.

Exemplos:

```
select * from graduacao.nota_aluno where id_aluno = '13403';
```



Cláusulas

ORDER BY

A cláusula ORDER BY é utilizada para ordenar uma tabela a partir de uma coluna ou um conjunto de colunas. É possível escolher a forma de ordenação dos seus dados, podem ser feitos de forma crescente(ASC) e decrescente(DESC), contudo se não for especificado o tipo de ordenação por padrão os dados serão organizados de forma crescente. Além disso, existe a possibilidade de escolher uma forma de ordenação para cada atributo do ORDER BY.

Um ponto importante sobre a utilização do ORDER BY é importante ter em mente que essa é uma operação custosa para o banco de dados dependendo do tamanho da base de dados, pois é feito um trabalho organização dos dados linha a linha, então como melhorar a utilização do ORDER BY? Solicitando ao DBA para criar um index nas colunas chaves da tabela, ou as colunas mais utilizadas para consultas, este index vai ajudar na ordenação dos dados e nas consultas a partir de filtros feitos no WHERE. Outra forma de melhorar o tempo de resposta da consulta ordenada é utilizando o WHERE para filtrar as informações antes da ordenação. Por esta razão é a última cláusula posicionada na sua query.



Cláusulas

Cláusulas - ORDER BY

Exemplos:

```
select * from graduacao.nota_aluno where id_aluno = '13403' order by ano asc;
```

```
select * from graduacao.nota_aluno order by id_aluno desc, ano asc;
```

LUIS_DEV



Cláusulas

Cláusulas - ORDER BY

Exemplos:

```
select * from graduacao.nota_aluno where id_aluno = '13403' order by ano asc;
```

```
select * from graduacao.nota_aluno order by id_aluno desc, ano asc;
```

LUIS_DEV



Cláusulas

GROUP BY

A cláusula GROUP BY, como o nome sugere, é que faz o agrupamento de registros a partir de funções agregadoras nas quais são selecionados um ou mais atributos de uma tabela para gerar um dataset de dados agregados. Quais são essas operações de agrupamento? Mínimo, máximo, contagem, total e média.

A cláusula GROUP BY como dito anteriormente pode ser feita com um ou mais atributos, mas quando feito o agrupamento com apenas uma coluna a cláusula GROUP BY é omitida, então não é preciso adicionar a cláusula na query. A partir da segunda coluna utilizada é necessário apontar na cláusula os atributos utilizados para o agrupamento. Importante destacar que o processo de agrupamento também pode ser oneroso ao banco dependendo da construção da query e quais tipos de operações utilizadas, então é aconselhável filtrar os dados, deixando apenas os dados que precisam ser agrupados e calculados.



Cláusulas

Cláusulas - GROUP BY

Exemplos:

```
select avg(total_creditos) from graduacao.aluno;
```

```
select max(total_creditos) from graduacao.aluno;
```

```
select min(total_creditos) from graduacao.aluno;
```

```
select count(id_aluno) from graduacao.aluno;
```

```
select count(distinct id_aluno) from graduacao.nota_aluno;
```

```
select count(id_aluno) from graduacao.nota_aluno;
```

```
select sum(total_creditos) from graduacao.aluno where  
nome_departamento = 'Pol. Sci.';
```

```
select nome_departamento, avg (salario) as media_salario
```

```
from graduacao.instrutor
```

```
group by nome_departamento;
```



Cláusulas

HAVING

A cláusula HAVING, é aplicada junto do agrupamento feito pelo GROUP BY, então quando há a necessidade de filtrar determinados grupos de dados gerados pelo GROUP BY e a partir disto se aplica a cláusula HAVING em cima de uma das funções de agregação existentes. Então, caso eu queira encontrar a lista de alunos com média de créditos acima de 50, HAVING permite esse filtro em cima dos dados já agrupados.

Exemplos:

```
select nome_departamento, avg (salario) as media_salario
```

```
from graduacao.instrutor
```

```
group by nome_departamento
```

```
having avg (salario) > 42000;
```



Operadores

Operadores matemáticos

Os operadores matemáticos tem como objetivo gerar operações básicas nas queries construídas.

Operadores:

- **soma(+)** - Soma de números
- **subtração(-)** - Subtrai valores
- **multiplicação(*)** - Multiplica números
- **divisão(/)** - Divide números
- **divisão(%)** - Divide números, mas retorna o resto da operação



Operadores

Operadores de comparação

Os operadores de comparação permitem com que valores sejam comparados gerando um retorno booleano como resultado da operação feita. Então, quando empregamos na cláusula WHERE os operadores filtram a partir dos predicados estabelecidos, logo há uma classificação de true e false para cada verificação a partir dos operadores. Retornando apenas o que for verdadeiro de acordo com o predicado construído na query.

Operadores:

- **(=)** - Compara se os valores de uma tabela são iguais dos valores utilizados para comparação e traz o resultado
- **(!= ou <>)** - Compara se valores da tabela são diferentes dos valores utilizados e traz o resultado do que for diferente
- **(>)** - Compara se os valores da tabela são maiores que valores utilizados para comparação e traz o resultado desse filtro
- **(<)** - Compara se os valores da tabela são menores que valores utilizados para comparação e traz o resultado desse filtro
- **(>=)** - Compara se os valores da tabela são maiores ou iguais aos valores utilizados para comparação e traz o resultado desse filtro
- **(<=)** - Compara se os valores da tabela são menores ou iguais aos valores utilizados para comparação e traz o resultado desse filtro



Operadores

Operadores lógicos

Os operadores lógicos permitem N condições sejam criadas a partir da utilização destes tipos de operadores, assim como os operadores de comparação os retornos são true, false e unknown. Na construção dos predicados do WHERE, os operadores lógicos são avaliados e geram o retorno de acordo com a combinação dos operadores e suas condições.

Operadores:

- **AND** - Verifica se todas as expressões são verdadeiras, o resultado do filtro é gerado caso todas as condições sejam verdadeiras, o AND é a primeira avaliação feita dentro do WHERE
- **OR** - Verifica se alguma das expressões são verdadeiras, o resultado do filtro é gerado casos alguma das condições seja verdadeira
- **BETWEEN** - Verifica se algum valor se enquadra inclusivamente dentro de um intervalo de valores definido
- **IN** - Verifica se algum valor especificado é igual a qualquer valor em uma subconsulta ou uma lista.
- **EXISTS** - Verifica se algum registro especificado está dentro do resultado da subconsulta no WHERE, isso respeitando a correlação dos atributos que filtram a subquery. Não se declara atributo a nível de comparação, precisa instanciar o EXISTS e a subconsulta.



Operadores

Operadores lógicos

Operadores:

- **NOT** - Operador que nega o resultado vindo de uma expressão construída a partir de uma condição estabelecida, então se o retorno for True é revertido para false e retornará tudo que for negação a verdade da condição. Pode ser utilizado também combinado com os operadores BETWEEN, EXISTS, LIKE, IS NULL e IN.
- **IS NULL** - Verifica se um valor é nulo
- **IS NOT NULL** - Verifica se um valor não é nulo

LUIS_DEV



Operadores

Operadores lógicos

Exemplo:

And

```
select * from graduacao.professor where ano = 2005  
and semestre = 'primavera';  
select * from graduacao.professor where ano = 2005  
and semestre = 'verão';
```

OR

```
select * from graduacao.professor where ano = 2005  
or semestre = 'primavera';  
select * from graduacao.professor where ano = 2005  
or semestre = 'verão';  
select * from graduacao.professor where ano = 2021  
or semestre = 'verão';
```

BETWEEN

```
select * from graduacao.aluno where total_creditos  
between 40 and 60 and nome_departamento = 'Civil  
Eng.';  
select * from graduacao.aluno where total_creditos  
not between 40 and 60;  
select * from graduacao.aluno where total_creditos  
not between 40 and 60 and nome_departamento =  
'Civil Eng.';  
select * from graduacao.aluno where total_creditos  
not between 40 and 60 or nome_departamento = 'Civil  
Eng.';
```

IN

```
select * from graduacao.aluno where total_creditos in  
(select total_creditos from graduacao.aluno where  
total_creditos > 100);
```



Operadores

Operadores lógicos

Exemplo:

EXISTS

```
select * from graduacao.aluno a where exists (select  
d.nome_departamento from graduacao.departamento  
d where d.nome_departamento =  
a.nome_departamento);
```

NOT /IS NULL/IS NOT NULL

```
select * from graduacao.aluno where total_creditos IS  
NOT NULL;  
select * from graduacao.aluno where total_creditos IS  
NULL;
```

LUIS_DEV



Operadores

Operador de texto

Tem como objetivo permitir a verificação de determinadas combinações de caracteres fazem parte uma string, não precisando ser um valor exato, estendendo as possibilidade de filtro além da utilização do comparador (=) no qual é preciso representar a string de busca corretamente ao que está na tabela, respeitando caracteres maiúsculos e minúsculos.

Operador:

- **LIKE-** Verifica se uma cadeia de caracteres está dentro de uma string preenchida em um atributo, utiliza o % como delimitador coringa na busca de caracteres.

Especificidades do LIKE:

- % no início da string pesquisada, busca resultados que terminem com a combinação de caracteres LIKE '%er'
- % no final da string pesquisada, busca resultados que terminem com a combinação de caracteres LIKE '%er'
- % no início e final da string pesquisada, busca resultados que esteja contido dentro da string contida no atributo com a combinação de caracteres LIKE '%car%'



Operadores

Operador de texto

Exemplo:

LIKE

```
select * from graduacao.aluno where nome_departamento like '%Eng.';  
select * from graduacao.aluno where nome_departamento like 'Pol%';  
select * from graduacao.aluno where nome_departamento like '%olo%';  
select * from graduacao.aluno where nome_departamento not like '%olo%' and  
total_creditos >= 50;
```



JOIN

O modelo de banco de dados relacional tem como objetivo principal gerar relacionamento entre suas tabelas a partir de atributos existentes entre elas, com isso cláusula JOIN traz a perspectiva de usar esse relacionamento entre as tabelas para gerar uma junção de dados através de atributos presentes nas relações nas quais queremos fazer as junções. O JOIN funciona, principalmente buscando por uma chave simples ou composta por elementos de mesma chave em outra relação, com isso gerando uma nova visão dos dados com novas colunas, com a mescla de colunas ou até mesmo gerando filtros de interseção ou exclusão dos dados no processo de junção. Importante lembrar que uma chave forte utilizada evita de gerar produto cartesiano a partir de uma junção, então na hora de modelar uma tabela é muito importante definir a chave que vai impedir a produção desse produto, levando em conta que o objetivo é classificar ou gerar visões únicas de cada registros. A cláusula JOIN utiliza o operador ON para possibilitar a comparação entre as chaves relacionais.

O SQL Server possui os seguintes tipos de JOIN:

- INNER JOIN
- LEFT [OUTER] JOIN
- RIGHT [OUTER] JOIN
- FULL [OUTER] JOIN
- CROSS JOIN



JOIN

INNER

O INNER JOIN faz com que a partir da junção apenas elementos de interseção contidos nas duas tabelas, então a comparação das chaves traz a mescla dos dados das duas tabelas.

Exemplo:

```
select A.*, B.nota,B.semestre from graduacao.aluno  
A inner join graduacao.nota_aluno B on a.id_aluno = b.id_aluno where a.id_aluno = 1000;
```

```
select A.*, B.nota,B.semestre, B.id_curso, C.titulo from graduacao.aluno A  
inner join graduacao.nota_aluno B on a.id_aluno = b.id_aluno  
inner join graduacao.curso C on C.id_curso = B.id_curso  
where a.id_aluno = 1000;
```



JOIN

LEFT

O LEFT JOIN assim como o INNER faz a junção de duas tabelas a partir dos mesmos atributos contidos nas tabelas, criando uma interseção, então a comparação das chaves traz retorna todos os elementos da tabela da esquerda com a adição dos atributos selecionados da tabela da direita.

Exemplo:

```
select
a.*,
b.nota,
b.semestre
from graduacao.aluno a
left join graduacao.nota_aluno b
on a.id_aluno = b.id_aluno ;
```

```
select
a.*,
b.nota,
b.semestre,
b.id_curso,
c.titulo
from graduacao.curso c
left join
graduacao.nota_aluno b
on c.id_curso = b.id_curso
left join
graduacao.aluno a
on a.id_aluno = b.id_aluno where b.nota is not null;
```



JOIN

LEFT

Exemplo:

```
select
a.*,
B.nota,
B.semestre
from graduacao.aluno a
left join
(select * from
graduacao.nota_aluno
where semestre = 'primavera') b
on a.id_aluno = b.id_aluno
where nota is null;
```

```
select
c.id_curso,
b.nota,
b.semestre
from graduacao.curso c
left join
graduacao.nota_aluno b
on C.id_curso = B.id_curso
where b.nota is null;
```

LUIS_DEV



JOIN

RIGHT

O RIGHT JOIN assim como o LEFT faz a junção de duas tabelas a partir dos mesmos atributos contidos nas tabelas, criando uma interseção, então a comparação das chaves traz retorna todos os elementos da tabela da direita com a adição dos atributos selecionados da tabela da esquerda.

Exemplo:

```
select
b.id_aluno,
c.id_orientador,
b.nome,
a.id_curso,
d.titulo
from graduacao.orientador_aluno c
right join
graduacao.aluno b
on c.id_aluno = b.id_aluno
left join
graduacao.nota_aluno a
on c.id_aluno = a.id_aluno
left join
graduacao.curso d
on a.id_curso = d.id_curso
where c.id_orientador is null;
```

```
select
c.id_curso,
b.nota,
b.semestre
from graduacao.nota_aluno b
right join
graduacao.curso c
on C.id_curso = B.id_curso where b.nota is null;
```



JOIN

FULL

O FULL JOIN permite que todas as linhas de dados da tabela à esquerda e da tabela à direita do JOIN retornem em um único dataframe. As linhas que possuem associação os valores de retorno serão baseados nos registros das duas tabelas. Caso contrário, se não houver associação a partir dos atributos selecionados, os valores das colunas serão nulos. O FULL JOIN é muito usado para achar registros órfãos entre as duas tabelas.

Exemplo:

```
select
*
from graduacao.orientador_aluno c
full join
graduacao.aluno b
on c.id_aluno = b.id_aluno
```

LUIS_DEV



JOIN

CROSS

O CROSS JOIN permite que todas as linhas de dados da tabela à esquerda e da tabela à direita do JOIN retornem em um único dataframe, criando um produto cartesiano de todas as possibilidades de cruzamento. Gerando uma associação rápida com todas as combinações entre duas ou mais tabelas.

Exemplo:

```
select
b.*,
c.*
from graduacao.nota_aluno b
cross join
graduacao.curso c;
```

```
select
b.*,
c.*
from graduacao.nota_aluno b
cross join
graduacao.curso c
where b.id_curso = c.id_curso;
```



JOIN

UPDATE

Podemos utilizar a cláusula a JOIN junto com a cláusula UPDATE com o objetivo de buscar uma determinada condição que só é possível a partir de uma junção e a partir disso gerar um filtro de atualização dos dados.

Exemplo:

```
update graduacao.instrutor  
set salario = salario *1.1  
from graduacao.instrutor a  
inner join graduacao.professor b  
on b.id_professor = a.id_instrutor  
where a.salario < 40000;
```

LUIS_DEV



JOIN

DELETE

Podemos utilizar a cláusula a JOIN junto com a cláusula DELETE com o objetivo de buscar uma determinada condição que só é possível a partir de uma junção e a partir disso gerar um filtro para exclusão dos dados.

Exemplo:

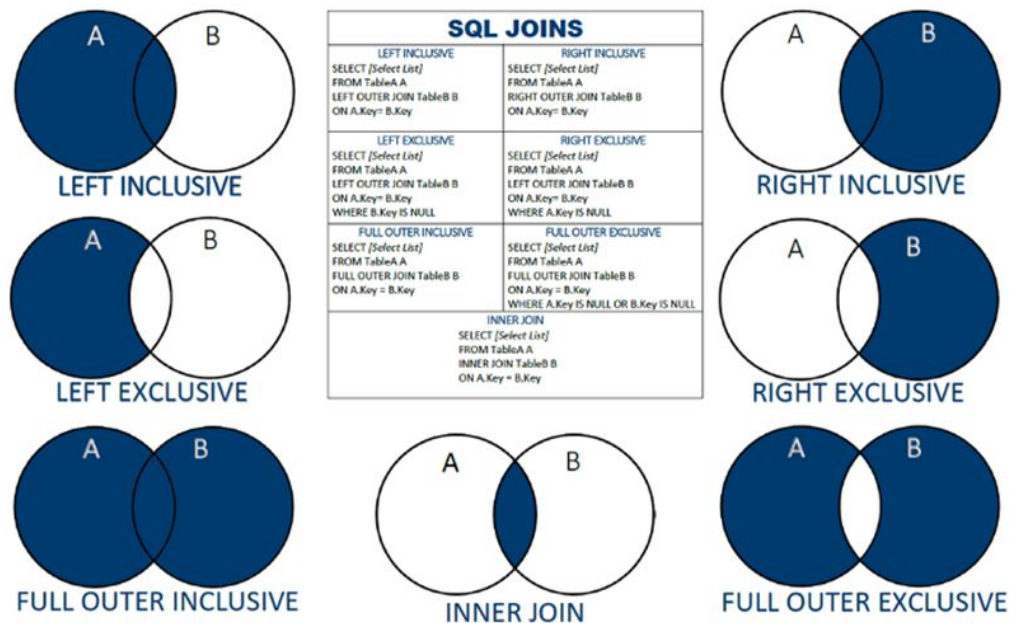
```
delete graduacao.instrutor
from graduacao.instrutor a
inner join graduacao.professor
b
on b.id_professor =
a.id_instrutor
where a.salario < 40000;
```

LUIS_DEV



JOIN

Compreensão gráfica das junções



Dúvidas?

LUIS_DEV

